

TDD 2025-12-Segundo Cuatr. TDD. $\rightarrow$ usar esta tecnica por la U3 que nos toca.

$\rightarrow$ tecnica practica de como implementamos y desarrollamos SW.

Test Driven Development (TDD)

es un Método para desarrollar SW conducido por pruebas

- lineamientos con cuestion de como queremos trabajar.

Ing
Diseño

① Requerimientos

② Analisis

③ Diseño

④ Implementación

Prueba

⑤ Testing

⑥ Despliegue

costo total
del proyecto

costo más representativo

horas personas lineales

COSTO DEL ESFUERZO

33+35

30-50

to martes semana que viene

dependen del SW

$\rightarrow$ el cliente se queda
gente de soporte al cliente

horas totales
sobre el proyecto
no de la programación

sobre

$\rightarrow$ Antes la gente (y actualmente) dejan las pruebas al final, muy tarde.

• No es necesario tener el SW terminado para hacer la prueba

$\rightarrow$ diseñar caso de prueba identificar las situaciones, + datos de ser necesario para poder identificar si uno o más criterios de aceptación se prueban

input para diseñar los escenarios

CASO DE PRUEBA $\rightarrow$ artefacto más importante

conjunto de situaciones que deben ser válidas

Tradicional

Diseñar $\rightarrow$ Desarrollar $\rightarrow$ Probar

adelantamos
trabajo

TDD

Diseño $\rightarrow$ Probar $\rightarrow$ Desarrollar

$\rightarrow$ los desarrolladores diseñan las pruebas unitarias antes de escribir el código

$\rightarrow$ el aseguramiento y control de calidad es más prod.

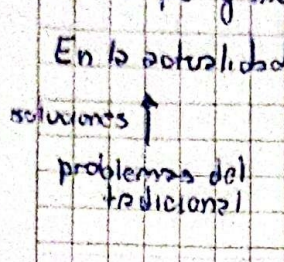
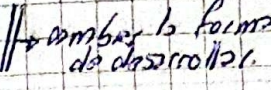
• COMO
PROBAR
EL SW
pensamiento

adelantarlo
para que el resto de ad
no consuman su tiempo
- el esfuerzo sigue

En el Agile surge las US con sus 3 partes, una de esta es las reglas de Prueba
 $\rightarrow$ adelantando todo lo que se pueda del pensamiento de la prueba

Asamblea

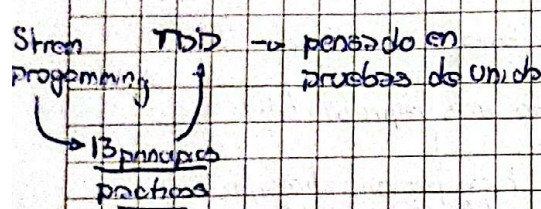
Culpar al testing y no aceptar su responsabilidad y función en la calidad destinando el tiempo y momento correcto.

En la actualidad  DepOps, el código que vas construyendo va siguiendo funcionando y atendiendo las necesidades del cliente de forma continua
Agile impulsa la idea de que en las primeras fases (Req - Anal - Diseño) pongamos el detalle donde es imprescindible
- requerimientos sin tanto detalle
- el detalle sea en los casos de prueba. 

Resuelto como probar estoy obligado a aprender como programarlo
PU -> caso de prueba -> + calidad de código
aseguro que el SW debe hacer lo que debe hacer.

VALIDAR
VERIFICAR

nivel 1
① Prueba de Unidad: prueba unitaria que la hace el propio desarrollador.

Stream programming  TDD -> pensado en pruebas de unidad
actualmente método de DESARROLLO
ATDD (Acceptance Test Driven Development)
desarrollo unificado a aceptación

Nota: Se debe probar todos los req FUNCIONA como NO

- ① Prueba Unitaria } componentes por separado
- ② Prueba Integración } funcionan como conjunto las pruebas unitarias, prueba de interfaces **Bloque**
- ③ Sistema o Version } ya es una versión, un build que cumple con los requerimientos TODOS Req
- ④ User Accept Test (UAT)

Atribuir lo más que se pueda al testing para mejorar la calidad del desarrollo.

• primero escribir casos de prueba que permiten crear el componente

CORE o P TDD

- el primer test debe fallar para poder mejorar e implementar.
con fallas no permite identificar.

YAGNI

- no escribir una línea de código hasta hacer un test caso que falle
- no es necesario más test de los necesarios para que falle
- no escribir más código en producción que el necesario para que el test pase.

Red
que falle el test

GREEN
que pase

Re-factor

Asamblea

REFACTORING Forma de desarrollar que apunta a mejorar la calidad interna del componente, no se cambia su funcionalidad solo que lo hago mejor

Reducir el nivel de acoplamiento y subir la cohesión, optimizar el componente.

+ TDD

principios de calidad básicos

no alcanza con que cumpla sino que sea lo más óptimo posible

prueba fail → código mínimo para que pase → pasa la prueba → Refactoring

por cada modificación se puede introducir error

Precondición estado inicial para que mi sistema pueda ejecutar una prueba en el contexto de la funcionalidad.
Activo todo lo que necesito disponible

+ los nombres de los métodos deben ser otros
def test_nombre_US_cumple_resultado (success/fail) ():
si es cierto está bien, pero debe de ser claro.

- indicar situación, el escenario para probar

Definir los pasos que reflejan las interacciones con el usuario con el sist y realizan el llamado a la US.
llamar la funcionalidad principal de la User Story y cambiar los parámetros

Resultado que se cumple con los resultados
assert: palabra clave que se utiliza para verificar condiciones en las pruebas.
Si la condición dentro del assert es False la prueba falla
assert == 1: implica que se deben de cumplir todas las condiciones para que se cumpla la prueba.
+ assert

Capturas los resultados

de el primer vistazo, que es lo que tengo, debo agregar o corregir.

Mensaje Final verdaderas, mensaje si el test pasa EXITO usando el escenario que estamos viendo.
si no paso ¿por que?

+ función específica a probar → pseudocódigo de la US.
+ test unitarios
un test por cada PU que debemos validar